

Informe Laboratorio 4

Sección 1

Alumno Eduardo Valenzuela
e-mail: eduardo@mail.udp.cl

Octubre de 2025

Índice

1. Descripción de actividades	2
2. Desarrollo de actividades según criterio de rúbrica	3
2.1. Investiga y documenta los tamaños de clave e IV	3
2.2. Solicita datos de entrada desde la terminal	4
2.3. Valida y ajusta la clave según el algoritmo	5
2.4. Implementa el cifrado y descifrado en modo CBC	8
2.5. Compara los resultados con un servicio de cifrado online	10
2.6. Describe la aplicabilidad del cifrado simétrico en la vida real	11

1. Descripción de actividades

Desarrollar un programa en Python utilizando la librería pycrypto para cifrar y descifrar mensajes con los algoritmos DES, AES-256 y 3DES, permitiendo la entrada de la key, vector de inicialización y el texto a cifrar desde la terminal.

Instrucciones:

1. Investigación

- Investigue y documente el tamaño en bytes de la clave y el vector de inicialización (IV) requeridos para los algoritmos DES, AES-256 y 3DES. Mencione las principales diferencias entre cada algoritmo, sea breve.

2. El programa debe solicitar al usuario los siguientes datos desde la terminal

- Key correspondiente a cada algoritmo.
- Vector de Inicialización (IV) para cada algoritmo.
- Texto a cifrar.

3. Validación y ajuste de la clave

- Si la clave ingresada es menor que el tamaño necesario para el algoritmo complete los bytes faltantes agregando bytes adicionales generados de manera aleatoria (utiliza `get_random_bytes`).
- Si la clave ingresada es mayor que el tamaño requerido, trunque la clave a la longitud necesaria.
- Imprima la clave final utilizada para cada algoritmo después de los ajustes.

4. Cifrado y Descifrado

- Implemente una función para cada algoritmo de cifrado y descifrado (DES, AES-256, y 3DES). Use el modo CBC para todos los algoritmos.
- Asegúrese de utilizar el IV proporcionado por el usuario para el proceso de cifrado y descifrado.
- Imprima tanto el texto cifrado como el texto descifrado.

5. Comparación con un servicio de cifrado online

- Selecciona uno de los tres algoritmos (DES, AES-256 o 3DES), ingrese el mismo texto, key y vector de inicialización en una página web de cifrado online.
- Compare los resultados de tu programa con los del servicio online. Valide si el resultado es el mismo y fundamente su respuesta.

6. Aplicabilidad en la vida real

- Describa un caso, situación o problema donde usaría cifrado simétrico. Defina que algoritmo de cifrado simétrico recomendaría justificando su respuesta.
- Suponga que la recomendación que usted entregó no fue bien percibida por su contraparte y le pide implementar hashes en vez de cifrado simétrico. Argumente cuál sería su respuesta frente a dicha solicitud.

2. Desarrollo de actividades según criterio de rúbrica

2.1. Investiga y documenta los tamaños de clave e IV

La investigación sobre los algoritmos de cifrado simétrico solicitados (DES, 3DES y AES-256) revela diferencias fundamentales en sus arquitecturas, principalmente en la longitud de la clave y el tamaño del bloque, lo cual impacta directamente en el tamaño requerido para el Vector de Inicialización (IV) [**HuntressIV**].

- **DES (Data Encryption Standard):** Es un algoritmo que funciona sobre bloques de 64 bits (8 bytes). Utiliza una clave de 64 bits, de los cuales 56 bits son para el cifrado y los 8 bits restantes para paridad. Dado que el tamaño del bloque es de 64 bits, el IV requerido para modos de operación como CBC (Cipher Block Chaining) también debe ser de 64 bits (8 bytes) [**StackExchangeDES**].
- **3DES (Triple DES):** Esta es una implementación que aplica el algoritmo DES tres veces para robustecer la seguridad. Si bien existen variantes, las implementaciones comunes utilizan claves de 112 bits o 168 bits efectivos y en su totalidad la clave tiene un tamaño de 192 bits. A pesar del aumento en la longitud de la clave, 3DES mantiene el tamaño de bloque original de DES, por lo que su IV sigue siendo de 64 bits (8 bytes) [**Wikipedia3DES**, **ASecuritySite3DES**].
- **AES-256 (Advanced Encryption Standard):** Es el estándar moderno de la industria. La variante AES-256 utiliza una clave de 256 bits (32 bytes), ofreciendo un alto nivel de seguridad. A diferencia de DES y 3DES, AES opera con un tamaño de bloque fijo de 128 bits (16 bytes), independientemente de la longitud de la clave. Por lo tanto, el IV para AES (en cualquier variante) en modo CBC es siempre de 128 bits (16 bytes) [**StackOverflowAES**].

La diferencia principal entre ellos radica en la seguridad y el rendimiento. DES se considera obsoleto y vulnerable a ataques de fuerza bruta. 3DES es significativamente más seguro que DES pero es computacionalmente lento [**Wikipedia3DES**]. AES-256 ofrece el nivel de seguridad más robusto de los tres, siendo el estándar para el cifrado de datos sensibles en la actualidad.

2.2. Solicita datos de entrada desde la terminal

Para cumplir con los requisitos de la pauta, se implementó la función principal `main` en Python, la cual actúa como la interfaz de entrada del programa. Esta función es responsable de solicitar al usuario tres datos clave antes de proceder con cualquier operación de cifrado: el tamaño de la clave (que determina el algoritmo a utilizar), la clave en sí misma y el vector de inicialización (IV).

El código fuente para esta captura de datos se presenta en el Listado 1.

```
def main():
    try:
        key_size_bits = int(input("Selecciona el tama o de la clave
                                   (64, 192 o 256 bits): "))
    except ValueError:
        print("Selecci n inv lida: debes ingresar un n mero
              entero.")
        return

    if key_size_bits not in SUPPORTED_CIPHERS:
        print("Tama o de clave no v lido. Debe ser 64, 192 o 256
              bits.")
        return

    raw_key = input("Introduce la clave (texto o hex): ")
    initialization_vector = input("Introduce el vector de
                                   inicializaci n (IV): ")
    plaintext = input("Introduce el texto plano: ")

    cipher_config = SUPPORTED_CIPHERS[key_size_bits]

    try:
        key, iv = validation_and_adjustment(cipher_config, raw_key,
                                             initialization_vector)
        print("Clave y IV validados correctamente.")
        demo_cipher(
            cipher_config["label"], cipher_config["encrypt"],
            cipher_config["decrypt"], plaintext, key, iv,
        )
    except ValueError as e:
        print(f"Error de validaci n: {e}")
        return
```

Listing 1: Función `main` para la captura de datos del usuario.

El flujo de ejecución del Listado 1 es el siguiente:

1. **Selección de Algoritmo:** Primero, se solicita al usuario que ingrese un tamaño de clave en bits (64 para DES, 192 para 3DES o 256 para AES-256).

2. **Validación de Entrada (Tipo):** El programa utiliza un bloque `try-except` para asegurar que la entrada sea un número entero. Si el usuario ingresa texto (ej: "sesenta"), el programa captura la excepción `ValueError`, notifica al usuario y termina la ejecución de forma segura.
3. **Validación de Entrada :** Inmediatamente después, se comprueba si el número ingresado (ej: 512) pertenece a la lista de algoritmos soportados (`SUPPORTED_CIPHERS`) que se encuentra hardcodeado. Si no es válido, se notifica al usuario y se termina la ejecución.
4. **Captura de Datos:** Una vez validado el algoritmo, el programa solicita la clave, el IV y el texto plano mediante la función `input()`. Es importante destacar que en este punto los datos se capturan como cadenas de texto. La validación y el ajuste de la longitud de estos datos (padding o truncamiento), se manejan en funciones posteriores.

2.3. Valida y ajusta la clave según el algoritmo

Para aplicar correctamente los algoritmos, fue necesario validar y ajustar la clave de entrada según los requisitos específicos de cada uno. Para gestionar esto, se definió un diccionario de configuración (`SUPPORTED_CIPHERS`, ver Listado 2) que centraliza los parámetros de los cifrados soportados. Este diccionario define la longitud de clave e IV en bytes, las funciones de cifrado/descifrado asociadas y requerimientos especiales, como el ajuste de paridad para DES y 3DES.

```
SUPPORTED_CIPHERS = {
    64: {
        "label": "DES",
        "key_length": 8,
        "iv_length": 8,
        "encrypt": encrypt_des,
        "decrypt": decrypt_des,
        "requires_parity": True,
    },
    192: {
        "label": "Triple DES",
        "key_length": 24,
        "iv_length": 8,
        "encrypt": encrypt_triple_des,
        "decrypt": decrypt_triple_des,
        "requires_parity": True,
    },
    256: {
        "label": "AES",
        "key_length": 32,
        "iv_length": 16,
        "encrypt": encrypt_aes,
```

```

        "decrypt": decrypt_aes,
    },
}

```

Listing 2: Diccionario de configuración de cifrados soportados.

Con esta configuración definida, se utilizaron las funciones de utilidad y ajuste presentadas en el Listado 3.

```

def _to_bytes(data):
    """Convierte una cadena en bytes, interpretando hex si es
    posible."""
    try:
        cleaned = data.replace(" ", "")
        return bytes.fromhex(cleaned)
    except ValueError:
        return data.encode("utf-8")

def _ensure_length(payload, length):
    """Asegura que el payload tenga la longitud exacta (bytes)."""
    payload = payload or b""
    if len(payload) < length:
        # Completa con bytes aleatorios si es m s corta
        payload += get_random_bytes(length - len(payload))
    elif len(payload) > length:
        # Trunca si es m s larga
        payload = payload[:length]
    return payload

def _adjust_des_parity(key_bytes):
    """Ajusta los bits de paridad para claves DES/3DES."""
    if len(key_bytes) % 8 != 0:
        raise ValueError("La clave DES debe tener multiples de 8
        bytes.")

    adjusted = bytearray()
    for byte in key_bytes:
        bits = byte & 0xFE # Limpia el bit de paridad (LSB)
        # Calcula la paridad impar requerida por DES
        parity = 1
        for shift in range(7):
            parity ^= (bits >> shift) & 1
        adjusted.append(bits | parity) # Aplica el bit de paridad
    return bytes(adjusted)

def validation_and_adjustment(cipher_config, key_input, iv_input):
    """Funci n principal que orquesta la validaci n y ajuste."""

```

```

base_key = _to_bytes(key_input)
key_bytes = _ensure_length(base_key, cipher_config["key_length"
])

if cipher_config.get("requires_parity", False):
    key_bytes = _adjust_des_parity(key_bytes)

iv_bytes = _ensure_length(_to_bytes(iv_input), cipher_config["
iv_length"])

key_bits = cipher_config["key_length"] * 8
print(f"Clave final ({key_bits} bits): {key_bytes.hex()}")
print(f"IV utilizado ({cipher_config['iv_length']} bytes): {
iv_bytes.hex()}")

return key_bytes, iv_bytes

```

Listing 3: Funciones de utilidad y ajuste de claves.

Explicación del funcionamiento

Estas funciones se encargan de tomar las entradas de texto del usuario y procesarlas para que sean válidas, siguiendo un orden específico por la función `validation_and_adjustment`:

1. **Conversión a Bytes (`_to_bytes`):** El primer paso es convertir la entrada del usuario (que es un *string*) a *bytes*. La función `_to_bytes` intenta primero interpretar la entrada como una cadena hexadecimal (útil si el usuario pega una clave existente). Si falla, asume que la entrada es texto plano y la codifica usando UTF-8.
2. **Ajuste de Longitud (`_ensure_length`):** Esta función implementa directamente los requisitos de la pauta.
 - Si la clave en bytes es **menor** que la longitud requerida (ej: 8 bytes para DES), completa los bytes faltantes agregando bytes aleatorios generados por `get_random_bytes`.
 - Si la clave es **mayor** que la longitud requerida, la trunca (corta) a la longitud necesaria, usando un *slice* de Python (`payload[:length]`).
3. **Ajuste de Paridad DES (`_adjust_des_parity`):** Este es un paso especial solo para DES y 3DES, indicado por la bandera `requires_parity` en el Listado 2. El estándar DES ignora el bit menos significativo (LSB) de cada byte de la clave y lo usa para paridad. Esta función calcula la paridad impar correcta para cada byte y la establece, asegurando que la clave sea válida para la librería de criptografía.
4. **Procesamiento del IV:** El IV pasa por el mismo proceso que la clave (pasos 1 y 2), pero omite el ajuste de paridad, ya que no es requerido.

5. **Impresión y Retorno:** Finalmente, la función imprime la clave y el IV finales en formato hexadecimal. Luego, retorna los `key_bytes` y `iv_bytes` listos para ser usados por las funciones de cifrado.

2.4. Implementa el cifrado y descifrado en modo CBC

Con los ajustes de clave e IV ya implementados, el siguiente paso es la creación de las funciones de cifrado y descifrado. Se desarrollaron seis funciones (tres para cifrar y tres para descifrar) que utilizan la librería `PyCryptodome`. Esta librería provee los objetos de cifrado (DES, DES3, AES) y el modo de operación `MODE_CBC`.

El código de estas funciones se presenta en el Listado 4.

```
# Se asume la importacion:
# from Crypto.Cipher import AES, DES, DES3
# from Crypto.Util.Padding import pad, unpad

# Encriptacion de los diferentes cifrados
def encrypt_des(plaintext, iv, key):
    cipher = DES.new(key, DES.MODE_CBC, iv)
    ciphertext = cipher.encrypt(pad(plaintext.encode("utf-8"), DES.
        block_size))
    return ciphertext

def encrypt_triple_des(plaintext, iv, key):
    cipher = DES3.new(key, DES3.MODE_CBC, iv)
    ciphertext = cipher.encrypt(pad(plaintext.encode("utf-8"), DES3.
        block_size))
    return ciphertext

def encrypt_aes(plaintext, iv, key):
    cipher = AES.new(key, AES.MODE_CBC, iv)
    ciphertext = cipher.encrypt(pad(plaintext.encode("utf-8"), AES.
        block_size))
    return ciphertext

# Desencriptacion de los diferentes cifrados
def decrypt_des(ciphertext, iv, key):
    cipher = DES.new(key, DES.MODE_CBC, iv)
    decrypted = unpad(cipher.decrypt(ciphertext), DES.block_size)
    return decrypted.decode("utf-8")

def decrypt_triple_des(ciphertext, iv, key):
    cipher = DES3.new(key, DES3.MODE_CBC, iv)
    decrypted = unpad(cipher.decrypt(ciphertext), DES3.block_size)
    return decrypted.decode("utf-8")
```



```
def decrypt_aes(ciphertext, iv, key):  
    cipher = AES.new(key, AES.MODE_CBC, iv)  
    decrypted = unpad(cipher.decrypt(ciphertext), AES.block_size)  
    return decrypted.decode("utf-8")
```

Listing 4: Funciones de cifrado y descifrado en modo CBC.

Explicación del funcionamiento

El Listado 4 implementa el flujo completo de cifrado y descifrado en modo CBC.

Proceso de Cifrado: Las tres funciones de cifrado siguen los mismos cuatro pasos:

1. **Instanciar el Cifrador:** Se crea un objeto de cifrado (ej: `AES.new`) al cual se le entregan tres parámetros: la clave ('key'), el modo de operación ('AES.MODE_CBC') y el vector de inicialización ('iv').
2. **Codificar a Bytes:** El texto plano (`plaintext`) es un *string* de Python. Dado que las operaciones criptográficas solo trabajan con bytes, se convierte usando `.encode('utf-8')`.
3. **Aplicar Relleno (Padding):** El modo CBC opera sobre bloques de tamaño fijo (8 bytes para DES/3DES, 16 para AES). Si el texto no es un múltiplo exacto de este tamaño, se corrompería. La función `pad()` (de `Crypto.Util.Padding`) añade los bytes de relleno necesarios siguiendo el estándar PKCS.
4. **Cifrar:** Finalmente, el método `.encrypt()` se aplica sobre los bytes codificados y rellenos, produciendo el texto cifrado ('ciphertext').

Proceso de Descifrado: El proceso inverso es simétrico y sigue estos cuatro pasos:

1. **Instanciar el Cifrador:** Se debe crear un objeto de cifrado idéntico al usado para cifrar, utilizando **exactamente la misma clave y el mismo IV**.
2. **Descifrar:** El método `.decrypt()` se aplica sobre el `ciphertext`. El resultado de esta operación son los bytes originales, pero **aún con el relleno (padding)**.
3. **Quitar Relleno (Unpadding):** La función `unpad()` se utiliza para inspeccionar los bytes descifrados, identificar el relleno PKCS y eliminarlo de forma segura.
4. **Decodificar a String:** Con los bytes originales puros ya recuperados, se utiliza `.decode('utf-8')` para convertirlos de nuevo en un *string* legible, que debe coincidir con el texto plano original.

2.5. Compara los resultados con un servicio de cifrado online

Para corroborar la correcta implementación de los algoritmos de cifrado, se utilizó la herramienta web CyberChef. Esta aplicación permitió validar los resultados del script de Python, utilizando el mismo texto plano, la misma clave y el mismo IV en modo CBC.

Se realizó la comprobación para los tres algoritmos, como se documenta en las Figuras 1, 2 y 3.

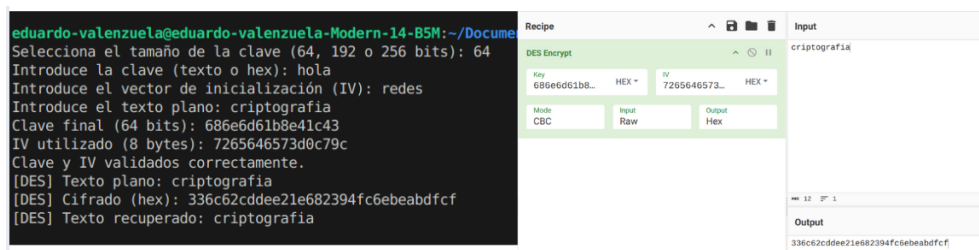


Figura 1: Comparativa de resultados DES.

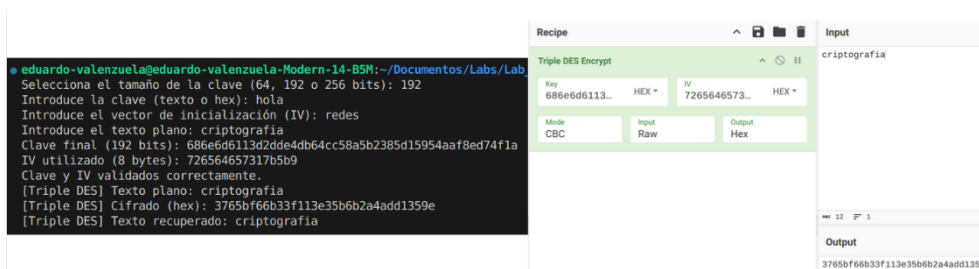


Figura 2: Comparativa de resultados 3DES.

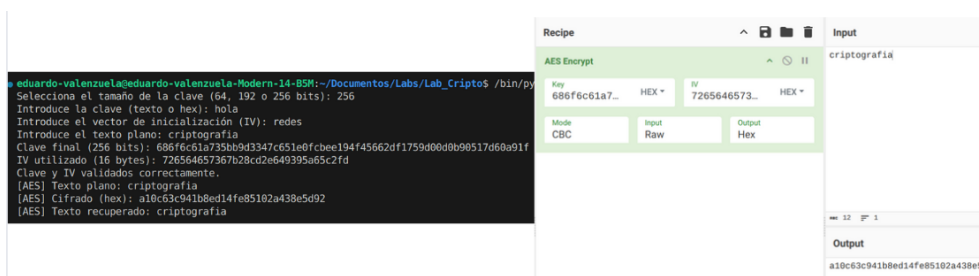


Figura 3: Comparativa de resultados AES-256.

Fundamentación del Resultado (AES-256)

Como se observa en la Figura 3, el resultado de la salida (el texto cifrado en formato hexadecimal) es **idéntico** entre el programa desarrollado en Python y la herramienta CyberChef.

Esta coincidencia valida la correcta implementación del programa:

- **Algoritmo y Modo:** Ambos utilizaron AES-256 en modo CBC (Cipher Block Chaining).
- **Clave (32 bytes):** Se utilizó la misma clave hexadecimal, correspondiente a la Clave finalimpresa en la terminal por el script (sección 2.3).
- **IV (16 bytes):** Se utilizó el mismo IV hexadecimal impreso por el script.
- **Relleno (Padding):** Este es un punto crucial. Ambas implementaciones (la librería PyCryptodome y CyberChef) utilizan por defecto el estándar de relleno **PKCS#7**. Si el relleno (padding) fuera diferente, el último bloque del texto cifrado no coincidiría.

Asimismo DES y triple DES coincidieron en los resultados.

2.6. Describe la aplicabilidad del cifrado simétrico en la vida real

El cifrado simétrico es ampliamente utilizado en la actualidad para la protección de datos sensibles, tanto a nivel empresarial como personal.

Caso de uso y recomendación

Un caso de uso fundamental es la **protección de comunicaciones en línea**, como las redes privadas virtuales (VPN) o las solicitudes HTTPS (que encapsulan protocolos como TLS/SSL). En estos escenarios, el cifrado simétrico se utiliza para proteger la confidencialidad de la información de los usuarios, como correos electrónicos, contraseñas y mensajes.

Al seleccionar un algoritmo, el menos recomendado sería **DES**, ya que está obsoleto y es altamente vulnerable a ataques por fuerza bruta.

Por el contrario, el algoritmo más recomendable es **AES-256**. La justificación de esta elección se basa en tres pilares principales:

- **Robustez:** Su clave de 256 bits ofrece una protección considerable contra ataques de fuerza bruta, siendo el estándar de seguridad más alto de los tres algoritmos analizados.
- **Rendimiento:** La rapidez es un factor crítico al seleccionar un algoritmo. AES es computacionalmente eficiente y muy superior a **3DES**, que es conocido por ser lento y costoso en términos de cómputo.
- **Adopción:** AES está ampliamente implementado y probado en la industria actual, lo que garantiza un gran soporte y confiabilidad.

Argumento: Cifrado vs. Hash

Ante la contrapropuesta de implementar *hashes* en lugar de cifrado simétrico, la respuesta debe fundamentarse en el objetivo del requerimiento.

Si bien el objetivo de un *hash* es proteger los datos para que no sean visibles en texto plano, se trata de una **función unidireccional**. Su uso principal es verificar la **integridad** de los datos; es decir, comprobar que un archivo o mensaje no ha sido alterado.

Sin embargo, si el objetivo es la **confidencialidad** la capacidad de cifrar información sensible y, posteriormente, descifrarla para su uso legítimo, el *hash* no es la herramienta adecuada. Para este fin, el cifrado simétrico es la opción correcta, y como se mencionó, AES-256 sería la principal propuesta.

Conclusiones y comentarios

En síntesis, a lo largo del presente laboratorio se estudió e implementó un conjunto de algoritmos de cifrado simétrico (DES, 3DES y AES-256) por medio de un código de Python.

Esta implementación práctica permitió al estudiante reconocer las diferencias fundamentales entre estos algoritmos y sus posibles aplicabilidades en la vida real. Como principal aprendizaje, se concluyó que algoritmos históricos como **DES** actualmente se encuentran obsoletos debido a su vulnerabilidad crítica frente a los ataques por fuerza bruta.

En contraparte, se analizó **AES-256** como uno de los algoritmos más seguros y ampliamente utilizados en la industria. Se comprendió que su robustez radica en la longitud de su clave (256 bits) y su eficiencia computacional, superando a alternativas más lentas como 3DES.

Además, se analizó la diferencia clave entre el cifrado y el *hashing*. Si bien ambos parecen tener el objetivo similar de ocultar la información en texto plano, se demostró que sus propósitos son muy dispares: el *hashing* se centra en la **integridad** de los datos (una función unidireccional), mientras que el cifrado protege la **confidencialidad** de estos (permitiendo la reversibilidad).

Referencias

- [1] Stack Exchange, "Does the initialization vector in DES have to be 8 bytes long?", *Cryptography Stack Exchange*, Accedido: 28 de octubre de 2025. [En línea]. Disponible en: <https://crypto.stackexchange.com/questions/76845/does-the-initialization-vector-in-des-have-to-be-8-bytes-long>
- [2] Huntress, "What is an Initialization Vector (IV)?", *Cybersecurity 101*, Accedido: 28 de octubre de 2025. [En línea]. Disponible en: <https://www.huntress.com/cybersecurity-101/topic/what-is-initialization-vector>
- [3] Stack Overflow, "What size of initialization vector needed for AES 256 encryption in Java?", Accedido: 28 de octubre de 2025. [En

- línea]. Disponible en: <https://stackoverflow.com/questions/31132162/what-size-of-initialization-vector-needed-for-aes-256-encryption-in-java>
- [4] Wikipedia, "Triple DES", *Wikipedia, The Free Encyclopedia*, Accedido: 28 de octubre de 2025. [En línea]. Disponible en: https://en.wikipedia.org/wiki/Triple_DES
- [5] A Security Site, "3DES", Accedido: 28 de octubre de 2025. [En línea]. Disponible en: https://asecuritysite.com/des/sym_3des
- [6] Repositorio del estudiante [En línea]. Disponible en: https://github.com/3dvvarMvb/Lab_Cripto/tree/main/Lab4